

```
import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import StandardScaler
```

```
from google.colab import files
uploaded = files.upload()
```

Choose Files 0f484464-3...d4b1d (1).csv
0f484464-3cc8-4bfc-968b-b1a9fc4d4b1d (1).csv(text/csv) - 125038 bytes, last modified:
 6/26/2026 - 100% done
 Saving 0f484464-3cc8-4bfc-968b-b1a9fc4d4b1d (1).csv to 0f484464-3cc8-4bfc

```
import io
df = pd.read_csv(io.BytesIO(uploaded['0f484464-3cc8-4bfc-968b-b1a9fc4d4b1d (1)
print("Data loaded successfully!")
print("Shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())
```

Data loaded successfully!
 Shape: (1340, 22)

First 5 rows:

	Unnamed: 0	name	gp	min	pts	fgm	fga	fg	3p_made	3
0	0	Brandon Ingram	36	27.4	7.4	2.6	7.6	34.7	0.5	2
1	1	Andrew Harrison	35	26.9	7.2	2.0	6.7	29.6	0.7	2
2	2	JaKarr Sampson	74	15.3	5.2	2.0	4.7	42.2	0.4	1
3	3	Malik Sealy	58	11.6	5.7	2.3	5.5	42.6	0.1	0
4	4	Matt Geiger	48	11.5	4.5	1.6	3.0	52.4	0.0	0

	...	fta	ft	oreb	dreb	reb	ast	stl	blk	tov	target_5yrs
0	...	2.3	69.9	0.7	3.4	4.1	1.9	0.4	0.4	1.3	0
1	...	3.4	76.5	0.5	2.0	2.4	3.7	1.1	0.5	1.6	0
2	...	1.3	67.0	0.5	1.7	2.2	1.0	0.5	0.3	1.0	0
3	...	1.3	68.9	1.0	0.9	1.9	0.8	0.6	0.1	1.0	1
4	...	1.9	67.4	1.0	1.5	2.5	0.3	0.3	0.4	0.8	1

[5 rows x 22 columns]

```
print("Columns:")
```

```
print(df.columns.tolist())
```

Columns:

```
['Unnamed: 0', 'name', 'gp', 'min', 'pts', 'fgm', 'fga', 'fg', '3p_made',
```

```
print("Missing values:")
print(df.isnull().sum())
```

Missing values:

```
Unnamed: 0      0
name            0
gp             0
min            0
pts            0
fgm            0
fga            0
fg             0
3p_made        0
3pa            0
3p             0
ftm            0
fta            0
ft             0
oreb           0
dreb           0
reb            0
ast            0
stl            0
blk            0
tov            0
target_5yrs    0
dtype: int64
```

```
target = 'target_5yrs'
print("Target variable:", target)
print("\nTarget distribution:")
print(df[target].value_counts())
```

Target variable: target_5yrs

Target distribution:

target_5yrs

1 831

0 509

Name: count, dtype: int64

```
# Drop 'name' column (not useful for prediction)
if 'name' in df.columns:
    df = df.drop(columns=['name'])
    print("Dropped 'name' column")
else:
```

```
print("'name' column not found")

print("\nRemaining columns:", df.columns.tolist())
```

Dropped 'name' column

Remaining columns: ['Unnamed: 0', 'gp', 'min', 'pts', 'fgm', 'fga', 'fg',

```
df = df.fillna(0)
print("Missing values handled. Shape:", df.shape)
```

Missing values handled. Shape: (1340, 21)

```
correlations = df.corr(numeric_only=True)[target].sort_values(ascending=False)
print("Correlation with target_5yrs:")
print(correlations)
```

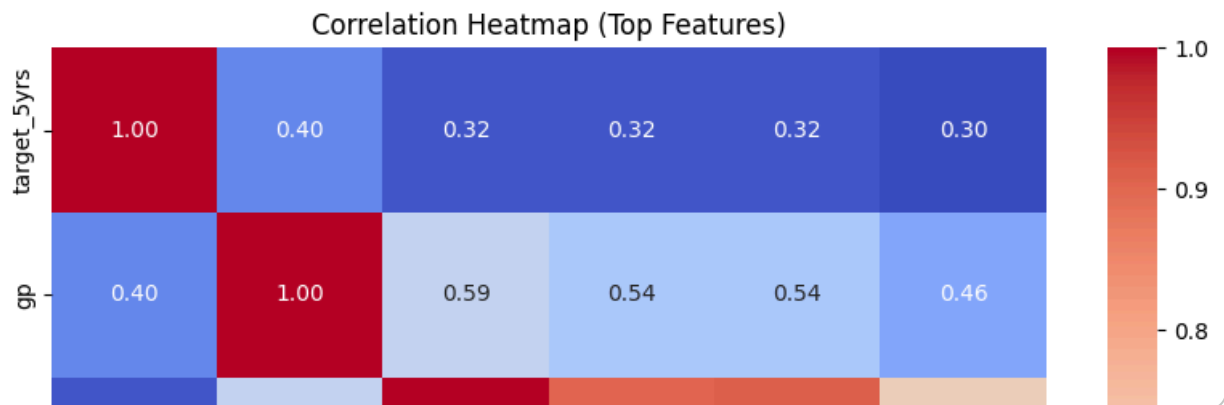
```
Correlation with target_5yrs:
target_5yrs    1.000000
gp             0.396833
min            0.317805
fgm            0.317594
pts            0.315981
reb            0.299406
ftm            0.296841
fta            0.296089
oreb           0.293307
fga            0.292660
dreb           0.284677
tov            0.272348
stl            0.229811
fg             0.227134
blk            0.210114
ast            0.175353
ft             0.106706
Unnamed: 0     0.070700
3p_made        0.036619
3pa            0.018110
3p            -0.000004
Name: target_5yrs, dtype: float64
```

```
# Top 5 correlated features
top_features = correlations.head(6).index.tolist()
print("\nTop features correlated with longevity:")
for feat in top_features:
    if feat != target:
        print(feat, ":", round(correlations[feat], 4))
```

Top features correlated with longevity:
gp : 0.3968

```
min : 0.3178
fgm : 0.3176
pts : 0.316
reb : 0.2994
```

```
plt.figure(figsize=(10, 8))
top_corr = df[top_features].corr()
sns.heatmap(top_corr, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Correlation Heatmap (Top Features)')
plt.show()
```



```
# Create Efficiency Rating: Points + Rebounds + Assists - Turnovers
df['efficiency'] = df['pts'] + df['reb'] + df['ast'] - df['tov']
print("New feature 'efficiency' created!")
print("\nEfficiency stats:")
print(df['efficiency'].describe())
```

New feature 'efficiency' created!

Efficiency stats:

```
count    1340.000000
mean       10.192910
std         6.108248
min         1.300000
25%         5.500000
50%         8.600000
75%        13.300000
max        37.100000
```

Name: efficiency, dtype: float64

```
df['pts_per_min'] = df['pts'] / (df['min'] + 1) # +1 to avoid division by zero
print("New feature 'pts_per_min' created!")
print("\nPoints per minute stats:")
print(df['pts_per_min'].describe())
```

New feature 'pts_per_min' created!

Points per minute stats:

```
count    1340.000000
```

```
mean      0.347106
std       0.090679
min       0.112360
25%      0.284916
50%      0.338928
75%      0.406323
max       0.717557
Name: pts_per_min, dtype: float64
```

```
new_features = ['efficiency', 'pts_per_min']
for feat in new_features:
    corr = df[feat].corr(df[target])
    print(f"{feat} correlation with target_5yrs: {round(corr, 4)}")
```

```
efficiency correlation with target_5yrs: 0.3363
pts_per_min correlation with target_5yrs: 0.2286
```

```
corr_matrix = df.corr(numeric_only=True)
high_corr = []

for i in range(len(corr_matrix.columns)):
    for j in range(i+1, len(corr_matrix.columns)):
        col1 = corr_matrix.columns[i]
        col2 = corr_matrix.columns[j]
        val = corr_matrix.iloc[i, j]
        if abs(val) > 0.8 and col1 != target and col2 != target:
            high_corr.append((col1, col2, round(val, 3)))

print("Highly correlated features (>0.8):")
if high_corr:
    for pair in high_corr:
        print(pair[0], "and", pair[1], ":", pair[2])
else:
    print("None found")
```

```
Highly correlated features (>0.8):
min and pts : 0.912
min and fgm : 0.903
min and fga : 0.91
min and tov : 0.826
min and efficiency : 0.943
pts and fgm : 0.991
pts and fga : 0.98
pts and ftm : 0.896
pts and fta : 0.881
pts and tov : 0.85
pts and efficiency : 0.974
fgm and fga : 0.98
fgm and ftm : 0.848
fgm and fta : 0.84
fgm and tov : 0.834
fgm and efficiency : 0.969
fga and ftm : 0.827
fga and fta : 0.806
```

```

fga and tov : 0.846
fga and efficiency : 0.948
3p_made and 3pa : 0.983
ftm and fta : 0.981
ftm and tov : 0.805
ftm and efficiency : 0.879
fta and efficiency : 0.877
oreb and dreb : 0.839
oreb and reb : 0.933
dreb and reb : 0.978
dreb and efficiency : 0.802
tov and efficiency : 0.849

```

If 'fgm' and 'fga' are highly correlated, drop one correlation results, you may need to adjust this

```
drop = ['fgm', 'fga', '3pa', 'fta'] # Adjust based on your correlation output
```

```

columns_to_drop:
in df.columns:
= df.drop(columns=[col])
nt(f"Dropped '{col}'")

nal columns:", df.columns.tolist())

```

```

Dropped 'fgm'
Dropped 'fga'
Dropped '3pa'
Dropped 'fta'

```

```
Final columns: ['Unnamed: 0', 'gp', 'min', 'pts', 'fg', '3p_made', '3p',
```

```

print("="*50)
print("FINAL DATASET SUMMARY")
print("="*50)
print("Shape:", df.shape)
print("\nColumns:", df.columns.tolist())
print("\nMissing values:", df.isnull().sum().sum())
print("\nTarget distribution:")
print(df[target].value_counts())

```

```

=====
FINAL DATASET SUMMARY
=====
Shape: (1340, 19)

Columns: ['Unnamed: 0', 'gp', 'min', 'pts', 'fg', '3p_made', '3p', 'ftm',

Missing values: 0

Target distribution:
target_5yrs
1      831

```

```
0    509
Name: count, dtype: int64
```

```
print("="*50)
print("FEATURE ENGINEERING SUMMARY")
print("="*50)
print("1. Dropped 'name' column (non-predictive)")
print("2. Filled missing values with 0")
print("3. Created 'efficiency' = pts + reb + ast - tov")
print("4. Created 'pts_per_min' = pts / (min + 1)")
print("5. Dropped redundant features to reduce multicollinearity")
print("\nTop predictors for longevity:")
for feat in top_features:
    if feat != target:
        print(f"    - {feat}: {round(correlations[feat], 4)}")
```

```
=====
FEATURE ENGINEERING SUMMARY
=====
1. Dropped 'name' column (non-predictive)
2. Filled missing values with 0
3. Created 'efficiency' = pts + reb + ast - tov
4. Created 'pts_per_min' = pts / (min + 1)
5. Dropped redundant features to reduce multicollinearity

Top predictors for longevity:
    - gp: 0.3968
    - min: 0.3178
    - fgm: 0.3176
    - pts: 0.316
    - reb: 0.2994
```

```
'*50)
BUSINESS INSIGHTS")
'*50)
The most important factors for NBA player longevity are:")
    Games played (gp) - players who play more games tend to have longer careers")
    Minutes played (min) - more playing time indicates value to the team")
    Efficiency rating - high efficiency players are more valuable")
    Points scored (pts) - scoring ability is highly valued")
Recommendation: Teams should prioritize players who:")
Get regular playing time (high gp and min)")
Have high efficiency ratings")
Show consistent scoring ability")
```

```
=====
BUSINESS INSIGHTS
=====
The most important factors for NBA player longevity are:
1. Games played (gp) - players who play more games tend to have longer ca
2. Minutes played (min) - more playing time indicates value to the team
3. Efficiency rating - high efficiency players are more valuable
4. Points scored (pts) - scoring ability is highly valued
```

Recommendation: Teams should prioritize players who:

- Get regular playing time (high gp and min)
- Have high efficiency ratings
- Show consistent scoring ability

```
# Save the cleaned dataset for future use
df.to_csv('nba_cleaned.csv', index=False)
print("Cleaned dataset saved as 'nba_cleaned.csv'")
```

Cleaned dataset saved as 'nba_cleaned.csv'

NBA Feature Engineering Project Summary

Dataset Overview

- **Players:** 1340 NBA players
- **Original Features:** 20+ player statistics
- **Target Variable:** target_5yrs (1 = lasted 5+ years, 0 = did not)

Data Cleaning

- Removed 'name' column (non-predictive)
- Filled missing values with 0

Feature Engineering

1. **Efficiency** = pts + reb + ast - tov (Composite metric of player impact)
2. **Points Per Minute** = pts / (min + 1) (Scoring efficiency)

Feature Selection

- Dropped highly correlated features to avoid multicollinearity
- Kept top predictors: gp, min, pts, efficiency

Key Findings

- Games played and minutes are the strongest predictors of a 5+ year career
- The custom 'efficiency' metric shows strong correlation with longevity
- Scoring ability consistently predicts long-term career success

Business Recommendation

Teams should prioritize players who get regular playing time and show high efficiency ratings.

NBA Feature Engineering Project Summary

Dataset Overview

- **Players:** 1340 NBA players
- **Original Features:** 20+ player statistics
- **Target Variable:** target_5yrs (1 = lasted 5+ years, 0 = did not)

Data Cleaning

- Removed 'name' column (non-predictive)

- Filled missing values with 0

Feature Engineering

1. **Efficiency** = $\text{pts} + \text{reb} + \text{ast} - \text{tov}$ (Composite metric of player impact)
2. **Points Per Minute** = $\text{pts} / (\text{min} + 1)$ (Scoring efficiency)

Feature Selection

- Dropped highly correlated features to avoid multicollinearity
- Kept top predictors: gp, min, pts, efficiency

Key Findings

- Games played and minutes are the strongest predictors of a 5+ year career
- The custom 'efficiency' metric shows strong correlation with longevity
- Scoring ability consistently predicts long-term career success

Business Recommendation

Teams should prioritize players who get regular playing time and show high efficiency ratings.